



VINUNIVERSITY

Lab 10: Shortest-path and Spanning Trees.

Week 13, May 18 2026

**College of Engineering and Computer Science,
Object-Oriented Programming & Data Structures Spring**

Lab Practice Submission Instructions:

- This is an individual lab practice and will typically be assigned in the computer laboratory.
- Your program should work correctly on all inputs. If there are any specifications about how the program should be written (or how the output should appear), those specifications should be followed.
- Your code and functions/modules should be appropriately commented. However, try to avoid making your code overly busy (e.g., include a comment on every line).
- Variables and functions should have meaningful names, and code should be organized into functions/methods where appropriate.
- Academic honesty is required in all work you submit to be graded. You should **NOT** copy or share your code with other students to avoid plagiarism issues.
- Use the template provided to prepare your solutions (modified to provide your student ID, your name, etc).
- You should **upload your .java file(s) to Codeforces before the end of the laboratory session** unless the instructor gave a specified deadline. **No Canvas or Codeforces submission** will get **0** point.
- Submit a separate .java file for each lab problem, with the exact filename specified in each lab question.
- Late submission of lab practice without an approved extension will incur the following penalties:
 1. No submission by the deadline will incur 0.25 point deduction for each problem (most of the problems are due at the end of the lab session).
 2. The instructor will deduct an additional 0.25 point per problem for each day past the deadline.
 3. The penalty will be deducted until the maximum possible score for the lab practice reaches zero (0%) unless otherwise specified by the instructor.
- **Navigate to the Lab:** Open the specific link provided for this week's lab: <https://vinuni26oop.contest.codeforces.com/>.

Problem 1: Campus Delivery Shortest Times

A campus delivery service needs to know the fastest travel time from the dispatch office to every building on campus.

The campus road network is an undirected weighted graph with non-negative travel times. Compute the shortest travel time from the `start` building to every other building using Dijkstra's algorithm.

Task Requirements

- Vertices are buildings labeled from `0` to `n-1`.
- Graph is undirected: each road connects both directions.
- Road travel times are non-negative.
- Print one line for every vertex in increasing order.

Input

- First line: `n m start`
- Next `m` lines: `u v w`

Where:

- `n`: number of buildings
- `m`: number of roads
- `start`: starting building
- `u, v`: endpoint buildings of a road
- `w`: travel time of that road

Output

For each vertex `i` from `0` to `n-1`, print:

- `i distance` if reachable
- `i INF` if unreachable

Examples

Example 1:

Input:

```
5 6 0
0 1 4
0 2 1
2 1 2
```

```
1 3 1
2 3 5
3 4 3
```

Output:

```
0 0
1 3
2 1
3 4
4 7
```

Example 2:

Input:

```
4 2 3
0 1 5
1 2 6
```

Output:

```
0 INF
1 INF
2 INF
3 0
```

Example 3:

Input:

```
5 6 1
1 0 0
1 2 2
0 2 1
2 3 0
3 4 4
0 4 10
```

Output:

```
0 0
1 0
2 1
3 1
4 5
```

Problem 2: Campus Delivery Actual Route

In Problem 1, the campus delivery service computed shortest travel times from the dispatch office to every building. Now a courier needs the actual sequence of roads to take to a specific destination.

Find the shortest path from start to target on a weighted undirected graph.

Input

- First line: `n m start target`
 - `n`: number of buildings (vertices, labeled 0 to `n-1`)
 - `m`: number of roads (edges)
 - `start`: source building
 - `target`: destination building
- Next `m` lines: `u v w` (road between `u` and `v` with travel time `w`)

Output

- If reachable (exactly two lines):

```
X
a b c
```

where `X` is the shortest distance and `a b c` are the path vertices in order.

- If unreachable: `-1`

Examples

Example 1:

Input:

```
5 6 0 4
0 1 2
0 2 5
1 2 1
1 3 2
2 4 3
3 4 1
```

Output:

```
5
0 1 3 4
```

Example 2:

Input:

```
4 2 0 3
0 1 4
1 2 5
```

Output:

-1

Example 3:

Input:

3 2 1 1

0 1 3

1 2 4

Output:

0

1

Problem 3: Campus Fiber Network Minimum Cost

VinUniversity wants to connect all campus buildings with fiber optic cables so that every building can access the internet. Buildings can communicate through intermediate buildings (e.g. if A connects to B and B connects to C, all three can communicate), so the installed cables must form a connected network.

Find the minimum total cost of cables needed to connect all buildings. If it is impossible to connect all buildings, output -1.

Input

- First line: two integers n m
 - n : number of buildings, labeled 0 to $n-1$
 - m : number of possible cables
- Next m lines: each line contains three integers u v w , meaning a cable between building u and building v with cost w

Output

- If all buildings can be connected: a single line with the minimum total cost.
- Otherwise: a single line with -1.

Examples

Example 1:

Input:

```
4 5
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
```

Output:

```
19
```

Example 2:

Input:

```
4 2
0 1 3
2 3 4
```

Output:

```
-1
```

Example 3:

 Input:

1 0

Output:

0

Problem 4: Campus Fiber Network — Unique Minimum Cost

In Problem 3, you computed the minimum total cost to connect all campus buildings. The engineering team now needs to know whether this minimum cost can be achieved by more than one distinct set of cables. If multiple different cable configurations all achieve the same minimum total cost, this gives the team flexibility during construction.

Hint: If there are multiple minimum spanning trees, then at least one edge in the tree can be replaced by another edge of equal weight without increasing the total cost.

Input

- First line: two integers n m
 - n : number of buildings, labeled 0 to $n-1$
 - m : number of possible cables
- Next m lines: each line contains three integers u v w , meaning a cable between building u and building v with cost w

Output

- If the graph is disconnected: -1
- If there is exactly one minimum spanning tree: YES
- If there are multiple distinct minimum spanning trees with the same total cost: NO

Examples

Example 1:

Input:

```
3 3
0 1 1
1 2 2
0 2 3
```

Output:

YES

Example 2:

Input:

```
4 2
0 1 3
2 3 4
```

Output:

-1

Example 3:

Input:

 1 0Output:
YES